Performing ANOVA in R

Jesse Wheeler

2026-09-02

Getting to Know the Data

Before running any models, it is crucial to understand the structure of your data.

```
# Load packages
library(dplyr)  # OPTIONAL

# Load the data
resin <- read.csv('resin.csv')

colnames(resin)
```

```
## [1] "temperature" "time"
```

```
dim(resin)
```

```
## [1] 37  2
```

```
head(resin)
```

```
##   temperature time
## 1         175 2.04
## 2         175 1.91
## 3         175 2.00
## 4         175 1.92
## 5         175 1.85
## 6         175 1.96
```

R tries to figure out the correct thing to do based on data type. In our lectures on ANOVA, we focused on computing means of **treatment group**:

$$y_{ij} \sim N\big(\mu, \sigma^2\big) \quad \text{vs} \quad y_{ij} \sim N\big(\mu_i, \sigma^2\big)$$

For this data, our response y_{ij} is the `time` variable, and the treatment is temperature.

```r
class(resin$temperature)
```

PROBLEM: temperature may be *numeric* type

```
## [1] "integer"
```

Because it is an `integer` type, R is going to treat this like a number for regression, **not** a treatment group / category. **This is a fairly common issue: treatment levels may be numbers, but we need to treat them like labels, not regression variables**.

```r
resin$temperature <- as.factor(resin$temperature)
class(resin$temperature)
```

Solution: Convert to a `factor`

```
## [1] "factor"
```

When a variable is a `factor` type, we can use the `table()` function to give categorical counts:

```r
table(resin$temperature)
```

```
##
## 175 194 213 231 250
##   8   8   8   7   6
```

The top row is the category (called a `level` in R), and the bottom row is the number of observations that are in that category, so we have $g = 5$ and

$$n_1 = 8,\ n_2 = 8,\ n_3 = 8,\ n_4 = 7,\ n_5 = 6$$

With the following treatment groups (`levels`):

```r
levels(resin$temperature)
```

```
## [1] "175" "194" "213" "231" "250"
```

Performing ANOVA with `aov()`

Now that we have computed the components manually, we can see how the `aov()` function does this all in one step. Compare the output below to the values we just calculated!

```r
# Run the built-in ANOVA function
model <- aov(time ~ temperature, data = resin)
summary(model)
```

```
##             Df Sum Sq Mean Sq F value Pr(>F)
## temperature  4  3.538  0.8844   96.36 <2e-16 ***
## Residuals   32  0.294  0.0092
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The Model and `as.factor()`

When conducting a Completely Randomized Design (CRD), our goal is to compare the **single mean model** (where all observations share one overall mean, μ) against the **separate group means model** (where each treatment group has its own mean, μ_i).

Here is the base code to run this in R:

```r
resin <- read.csv('03/resin.csv')
model <- aov(time ~ as.factor(temperature), data = resin)
summary(model)
```

Why `as.factor()` is critical: If your treatment levels are numeric (e.g., temperatures of 150, 175, 200), R will default to treating them as a continuous variable. It will attempt to fit a single regression line rather than estimating separate categorical group means (μ_i). Wrapping the variable in `as.factor()` forces R to treat each temperature as a distinct, independent treatment group ($i = 1, \ldots, g$).

Mapping the R Output to ANOVA Mechanics

Running `summary(model)` produces the classic ANOVA table. This table directly computes the Sum of Squares (SS) and Mean Squares (MS) used to compare the fit of the reduced model versus the full model.

R Output Column	Slide Notation	Interpretation
Df	$g - 1$ and $N - g$	Degrees of freedom for treatments and error, respectively.
Sum Sq	SS_{Trt} and SS_E	SS_{Trt} (Temperature row) measures variance *between* group means. SS_E (Residuals row) measures variance *within* groups.
Mean Sq	MS_{Trt} and MS_E	The Sum of Squares divided by their respective Degrees of Freedom. MS_E is our unbiased estimate of the variance, $\hat{\sigma}^2$.
F value	F	MS_{Trt}/MS_E. A larger F indicates the separate means model explains significantly more variance than the single mean model.

R Output Column	Slide Notation	Interpretation
Pr(>F)	p-value	The probability of seeing this F-statistic if the single mean model ($H_0 : \alpha_i = 0$) were true.

Estimating Parameters: Means and Treatment Effects

Your slides note that any group mean can be written as $\mu_i = \mu^* + \alpha_i$, where μ^* is the overall mean and α_i is the specific treatment effect.

1. Finding the Group Means ($\hat{\mu}_i$)

To find the separate group means ($\bar{y}_{i\bullet}$), we can aggregate the data:

```
# Returns the mean 'time' for each 'temperature' group
tapply(resin$time, resin$temperature, mean)
```

2. Finding the Treatment Effects ($\hat{\alpha}_i$)

Software handles the overall mean (μ^*) in different ways. By default, the `aov()` function paired with `model.tables()` uses the **weighted average** approach for the overall mean ($\hat{\mu}^* = \bar{y}_{\bullet\bullet}$), where $\sum n_i \hat{\alpha}_i = 0$.

```
# Computes the estimated treatment effects (alpha_hat) for each group
model.tables(model, type = "effects")
```

Note on Reference Groups: If you instead run `lm(time ~ as.factor(temperature))`, R defaults to the **Reference Group** parameterization ($\hat{\mu}^* = \bar{y}_{1\bullet}$). In that summary, the intercept is the mean of the first group, and the coefficients are the treatment effects ($\hat{\alpha}_i$) representing the difference between group i and the first group.

---

2. Computing ANOVA "By-Hand" in R

To understand the formulas from the lecture, we can calculate the Sum of Squares (SS) and Mean Squares (MS) using basic R functions before letting `aov()` do the heavy lifting.

Step A: Calculate the Means and Treatment Effects

Recall that the treatment effect is $\hat{\alpha}_i = \bar{y}_{i\bullet} - \bar{y}_{\bullet\bullet}$.

```
# 1. Grand Mean (y_bar_dot_dot)
grand_mean <- mean(resin$time)

# 2. Group Means (y_bar_i_dot)
group_means <- tapply(resin$time, resin$temperature, mean)
```

```r
# 3. Treatment Effects (alpha_hat_i)
alpha_hat <- group_means - grand_mean

print(alpha_hat)
```

```
##         175         194         213         231         250
##  0.46736486  0.16361486 -0.08763514 -0.27084942 -0.40846847
```

Step B: Calculate the Sum of Squares

We will partition the variance into Total (SS_T), Treatment (SS_{Trt}), and Error (SS_E).

```r
# Total Sum of Squares (SS_T): deviation of every point from the grand mean
SS_T <- sum((resin$time - grand_mean)^2)

# Treatment Sum of Squares (SS_Trt): variance explained by the groups
# Formula: sum(n_i * alpha_hat^2)
n_i <- table(resin$temperature)
SS_Trt <- sum(n_i * (alpha_hat)^2)

# Error Sum of Squares (SS_E): what's left over (SS_T = SS_Trt + SS_E)
SS_E <- SS_T - SS_Trt

# We can also calculate SS_E directly by finding the variance within each group
# group_vars <- tapply(resin$time, resin$temperature, var)
# SS_E_direct <- sum((n_i - 1) * group_vars)
```

Step C: Calculate the F-Statistic

To find the F-statistic, we divide our Sum of Squares by their respective degrees of freedom to get the Mean Squares (MS).

```r
# Number of groups (g)
g <- length(levels(resin$temperature))
N <- nrow(resin)

# Degrees of Freedom
df_trt <- g - 1
df_error <- N - g

# Mean Squares
MS_Trt <- SS_Trt / df_trt
MS_E <- SS_E / df_error

# F-Statistic
F_stat <- MS_Trt / MS_E
print(paste("Calculated F-Statistic:", round(F_stat, 3)))
```

```
## [1] "Calculated F-Statistic: 96.363"
```
